

FlashDrop Backend fue refactorizado de un monolito Node.js a 4 microservicios Spring Boot con arquitectura hexagonal. En esta primera etapa, los 4 servicios comparten la **misma base de datos Supabase**. El objetivo de esta migración es que **cada microservicio tenga su propia base de datos independiente**, eliminando todo acoplamiento a nivel SQL entre servicios.

IMPORTANTE: No hay datos en producción, por lo tanto podemos crear las bases de datos nuevas desde cero con esquemas limpios. No se requiere migración de datos en vivo.

Cada tabla del esquema actual queda asignada a **un único microservicio dueño**. Ningún otro servicio puede leer ni escribir esas tablas directamente.

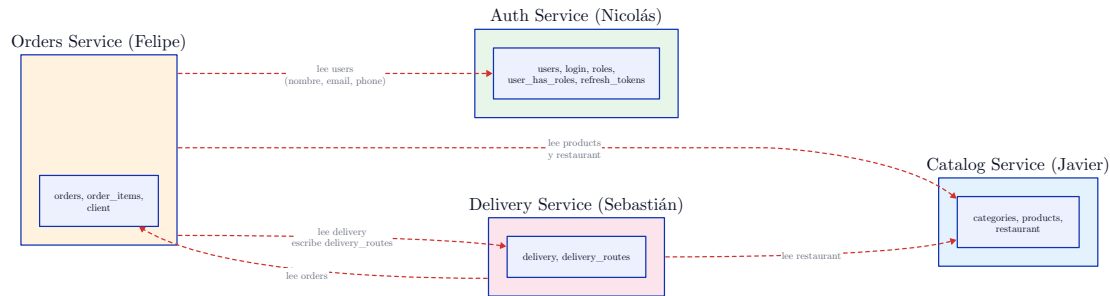
Tabla	Dueño	Responsable	Justificación
users	Auth Service	Nicolás	Identidad del usuario, datos de perfil
login	Auth Service	Nicolás	Credenciales de autenticación
roles	Auth Service	Nicolás	Definición de roles del sistema
user_has_roles	Auth Service	Nicolás	Asignación usuario ↔ rol
refresh_tokens	Auth Service	Nicolás	Tokens de refresco (tabla nueva creada por auth)
categories	Catalog Service	Javier	Categorías de productos
products	Catalog Service	Javier	Productos del catálogo
restaurant	Catalog Service	Javier	Restaurantes (como entidad del catálogo comercial)
orders	Orders Service	Felipe	Pedidos
order_items	Orders Service	Felipe	Ítems de cada pedido
client	Orders Service	Felipe	Perfil de cliente (vinculado a pedidos)
delivery	Delivery Service	Sebastián	Perfil de repartidor

Tabla	Dueño	Responsable	Justificación
delivery_routes	Delivery Service	Sebastián	Rutas de entrega

⚠ **ATENCIÓN:** La tabla `restaurant` es la más disputada. Aparece en Catalog (para listar restaurantes y sus productos), en Orders (para asociar pedidos), y en Delivery (para obtener la dirección de retiro). **El dueño es Catalog** porque es donde se gestiona la entidad comercial. Los demás servicios deben consultar a Catalog por API.

Estas son las consultas SQL que actualmente un servicio hace contra tablas que **no le pertenecen**. Cada una debe ser reemplazada por una llamada HTTP al servicio dueño.

Diagrama de dependencias actuales



2.1 Orders Service (Felipe) → Tablas ajenas

Archivo	Tabla que consulta	Dueño real	Qué necesita
SupabaseRestCatalogAdapter.java	products	Catalog (Javier)	Obtener productos por IDs (precio, nombre, disponibilidad)
SupabaseRestCatalogAdapter.java	restaurant	Catalog (Javier)	Obtener restaurante por ID y por user_id
SupabaseRestDeliveryAdapter.java	users	Auth (Nicolás)	Obtener nombre, email, teléfono del cliente/repartidor

Archivo	Tabla que consulta	Dueño real	Qué necesita
SupabaseRestDeliveryAdapter.java	delivery	Delivery (Sebastián)	Obtener delivery_id por user_id, info del repar-tidor
SupabaseRestOrderRepositoryAdapter.java	delivery_routes	Delivery (Sebastián)	Crear y actualizar rutas de entrega

2.2 Delivery Service (Sebastián) → Tablas ajenas

Archivo	Tabla que consulta	Dueño real	Qué necesita
OrderServiceClientAdapter.java	orders	Orders (Felipe)	Obtener órdenes por IDs (dirección, restaurant_id)
OrderServiceClientAdapter.java	restaurant	Catalog (Javier)	Obtener nombre y dirección del restaurante

2.3 Catalog Service (Javier) → Tablas ajenas

✓ **Limpio.** Catalog solo accede a `categories`, `products` y `restaurant` (todas propias).

2.4 Auth Service (Nicolás) → Tablas ajenas

✓ **Limpio.** Auth solo accede a `users`, `login`, `roles`, `user\_has\_roles` y `refresh\_tokens` (todas propias).

Para que los servicios que hoy consultan tablas ajenas puedan obtener la misma información vía HTTP, los servicios dueños deben exponer estos endpoints **internos**:

3.1 Nicolás — Auth Service

```
GET \api/internal/users/{userId}
→ Retorna: { id, name, lastName, email, phone }

GET \api/internal/users/{userId}/roles
→ Retorna: \[{ id, name }\]
```

3.2 Javier — Catalog Service

```
GET \api/internal/products?ids\={id1,id2,...}
→ Retorna: \[{ id, restaurantId, name, description, price, image, isAvailable }\]
```

```
GET \api/internal/restaurants/{restaurantId}
→ Retorna: { id, userId, name, address }
```

```
GET \api/internal/restaurants?userId\={userId}
→ Retorna: { id, userId, name, address }
```

3.3 Felipe — Orders Service

```
GET \api/internal/orders?ids\={id1,id2,...}
→ Retorna: \[{ id, clientId, restaurantId, deliveryId, status, address }\]
```

3.4 Sebastián — Delivery Service

```
GET \api/internal/delivery-persons?userId\={userId}
→ Retorna: { id, userId, vehicle }
```

```
POST \api/internal/routes
→ Body: { orderId, pickupAddress, deliveryAddress, distanceKm, estimatedMinutes,
status }
```

```
PATCH \api/internal/routes/{orderId}/status
→ Body: { status }
```

 **TIP:** Los endpoints internos (`\api/internal/`) deben estar protegidos para que solo sean accesibles entre microservicios (por red interna, API key compartida, o JWT de servicio). No deben exponerse al cliente público.

4.1 Nicolás — Auth Service

Auth ya está limpio. Solo accede a sus propias tablas.

- [] **Crear endpoints internos** (`\api/internal/users/{id}` y `\api/internal/users/{id}/roles`) para que otros servicios consulten datos de usuario sin acceder a la tabla `users` directamente.
- [] **Crear esquema SQL independiente** con las tablas: `users`, `login`, `roles`, `user\has\roles`, `refresh\tokens`.
- [] **Configurar nueva conexión** a la base de datos propia de Auth en `application.yml`.
- [] **Crear script de seed** con datos demo para la BD independiente de Auth.

4.2 Felipe — Orders Service

⚠ ATENCIÓN: Orders es el servicio con MÁS dependencias cruzadas. Lee tablas de Auth, Catalog Y Delivery.

- [] **Reemplazar `SupabaseRestCatalogAdapter`**: en vez de consultar las tablas `products` y `restaurant` directamente, crear un `CatalogHttpClientAdapter` que implemente `CatalogPort` y llame a los endpoints internos del Catalog Service.
- [] **Reemplazar accesos a `users` en `SupabaseRestDeliveryAdapter`**: en vez de leer la tabla `users`, llamar al endpoint interno de Auth Service (`\api/internal/users/{id}`).
- [] **Reemplazar accesos a `delivery` en `SupabaseRestDeliveryAdapter`**: en vez de leer la tabla `delivery`, llamar al endpoint interno de Delivery Service (`\api/internal/delivery-persons?userId={id}`).
- [] **Mover escritura de `delivery\_routes`**: el `OrderRepositoryAdapter` actualmente crea y actualiza `delivery\_routes`. Esa tabla es de Delivery Service. Opciones:
- **Opción A (recomendada)**: Orders llama al endpoint interno de Delivery `POST \api/internal/routes` para crear la ruta.
- **Opción B**: Si se necesita actualizar estado de ruta, usar `PATCH \api/internal/routes/{orderId}/status`.
- [] **Crear esquema SQL independiente** con las tablas: `orders`, `order\_items`, `client`.
- [] **Configurar nueva conexión** a la base de datos propia de Orders.
- [] **Crear script de seed** con datos demo.
- [] **Agregar configuración** de URLs de los servicios internos (auth-service, catalog-service, delivery-service) en `application.yml`.

4.3 Javier — Catalog Service

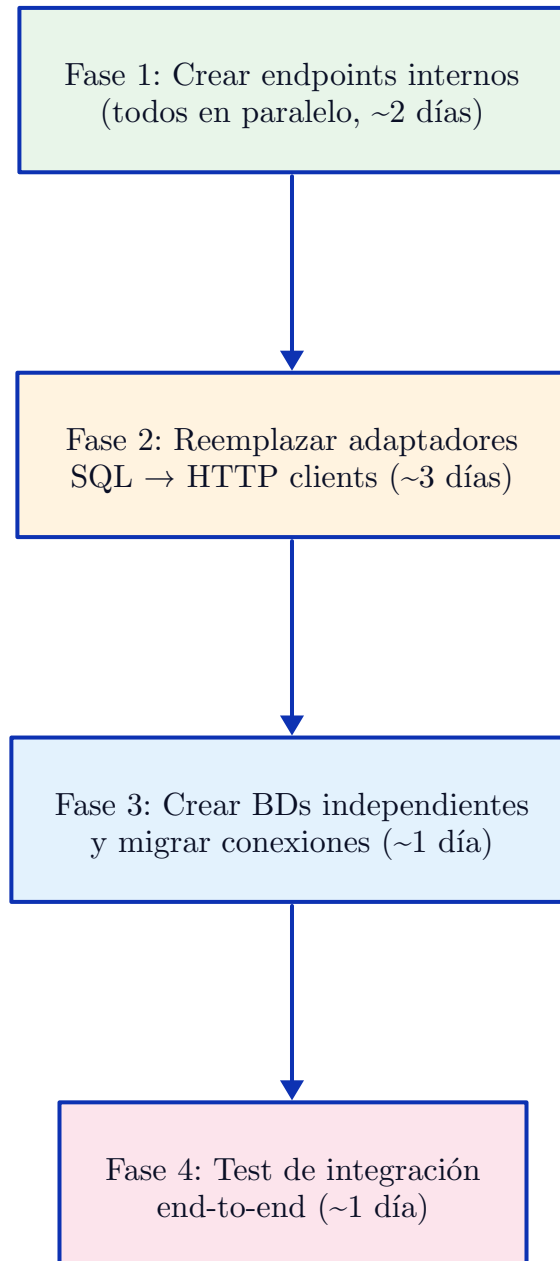
Catalog ya está limpio. Solo accede a sus propias tablas.

- [] **Crear endpoints internos** para que Orders y Delivery consulten productos y restaurantes.
- [] **Crear esquema SQL independiente** con las tablas: `categories`, `products`, `restaurant`.
- [] **Configurar nueva conexión** a la base de datos propia de Catalog.
- [] **Crear script de seed** con datos demo (categorías, productos, restaurantes).

4.4 Sebastián — Delivery Service

- [] **Reemplazar `OrderServiceClientAdapter`**: actualmente consulta las tablas `orders` y `restaurant` directamente vía Supabase REST. Debe llamar a los endpoints internos de Orders Service y Catalog Service.
 - [] **Crear endpoints internos** (`\api/internal/delivery-persons`, `\api/internal/routes`) para que Orders pueda consultar/crear rutas y buscar repartidores.
 - [] **Crear esquema SQL independiente** con las tablas: `delivery`, `delivery\_routes`.
 - [] **Configurar nueva conexión** a la base de datos propia de Delivery.
 - [] **Crear script de seed** con datos demo.
-

Las tareas tienen dependencias. Este es el orden que minimiza bloqueos entre el equipo:



Fase 1 — Endpoints internos (todos en paralelo, 2 días)

Cada persona crea los endpoints internos de su servicio. No se toca nada más. Al final de esta fase, todos los endpoints internos están disponibles y testeados.

Quién	Qué hace
Nicolás	Crea <code>GET \api/internal/users/{id}</code> y <code>GET \api/internal/users/{id}/roles</code>
Javier	Crea <code>GET \api/internal/products?ids\=...</code> , <code>GET \api/internal/restaurants/{id}</code> , <code>GET \api/internal/restaurants?userId\=...</code>
Sebastián	Crea <code>GET \api/internal/delivery-persons?userId\=...</code> , <code>POST \api/internal/routes</code> , <code>PATCH \api/internal/routes/{orderId}/status</code>
Felipe	Crea <code>GET \api/internal/orders?ids\=...</code> (para Delivery)

Fase 2 — Reemplazar adaptadores (dependencia: Fase 1, 3 días)

Cada servicio reemplaza sus adapters que acceden tablas ajenas por HTTP clients que llaman a los endpoints creados en Fase 1. **La BD sigue siendo compartida** pero ya nadie accede a tablas ajenas.

Quién	Qué hace
Felipe	Reemplaza <code>SupabaseRestCatalogAdapter</code> → <code>CatalogHttpClientAdapter</code> , reemplaza accesos a <code>users</code> y <code>delivery</code> por llamadas a Auth y Delivery
Sebastián	Reemplaza <code>OrderServiceClientAdapter</code> para que llame a Orders API y Catalog API en vez de Supabase directo

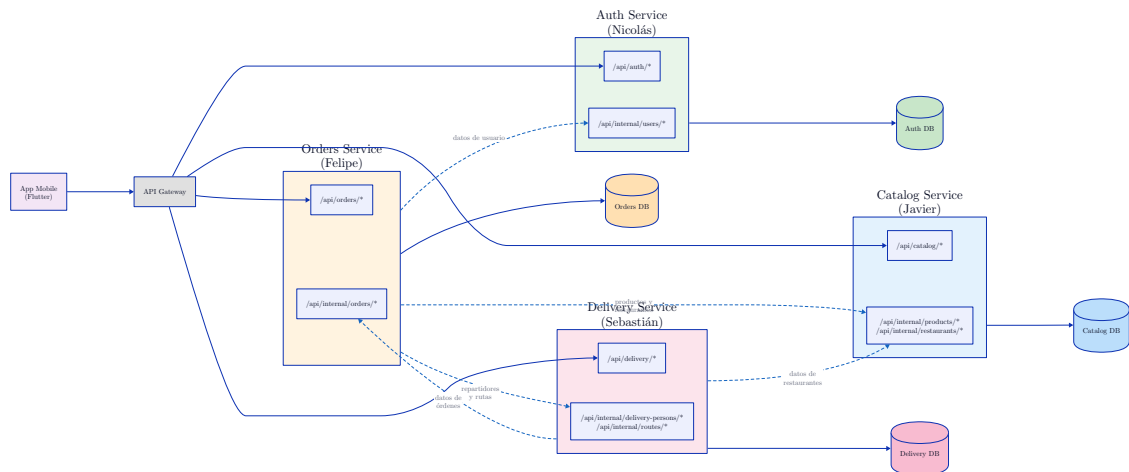
Fase 3 — Separar bases de datos (1 día)

Una vez validado que ningún servicio accede a tablas ajenas:

- [] Crear 4 proyectos/instancias de Supabase (o 4 esquemas PostgreSQL con usuarios independientes)
- [] Cada servicio tiene su propio `SUPABASE\_URL` y `SUPABASE\_SERVICE\_ROLE\_KEY`
- [] Ejecutar los scripts de seed en cada BD independiente
- [] Actualizar los `.env` de cada microservicio

Fase 4 — Validación end-to-end (1 día)

- [] Levantar los 4 servicios apuntando a sus BDs independientes
 - [] Probar flujo completo: registro → login → ver catálogo → crear orden → asignar repartidor → actualizar estado de ruta
 - [] Verificar que NO haya queries fallidas por tablas faltantes
-



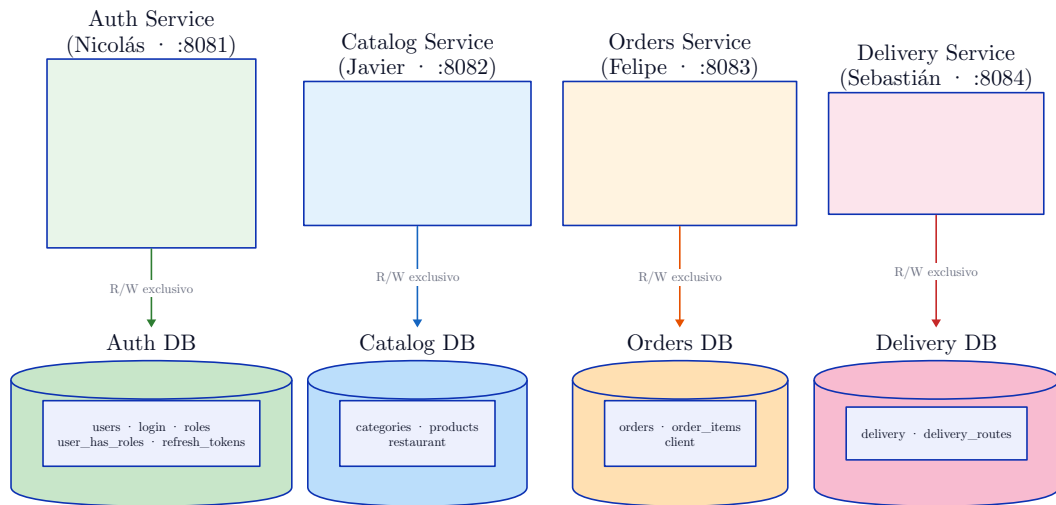
La arquitectura hexagonal ya preparó el terreno. Los ports (interfaces) que necesitamos ya están definidos:

- **CatalogPort** (Orders) — Ya define la interfaz. Solo hay que cambiar la implementación de SQL a HTTP.
- **DeliveryPort** (Orders) — Mismo caso.
- **OrderServicePort** (Delivery) — Ya tiene el port definido.

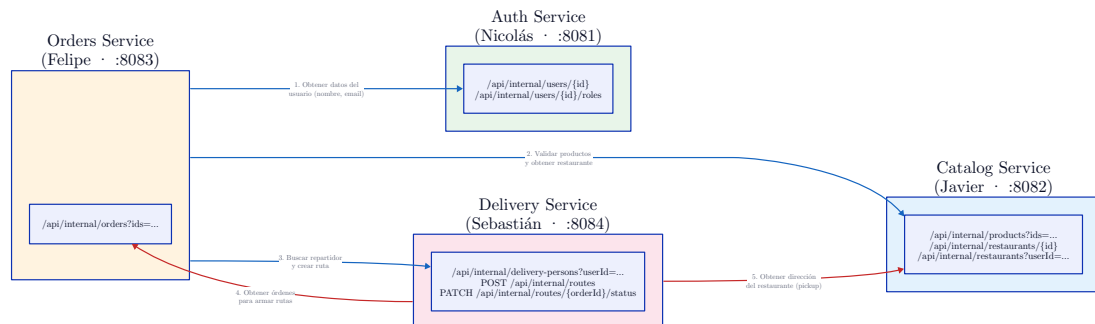
Los casos de uso y la lógica de dominio NO se tocan. Solo cambian los adaptadores de infraestructura, que es EXACTAMENTE para lo que sirve la arquitectura hexagonal.

Este diagrama muestra **cada operación concreta** que cada servicio realiza: lecturas/escrituras a su propia BD y llamadas HTTP a otros servicios, con los endpoints específicos.

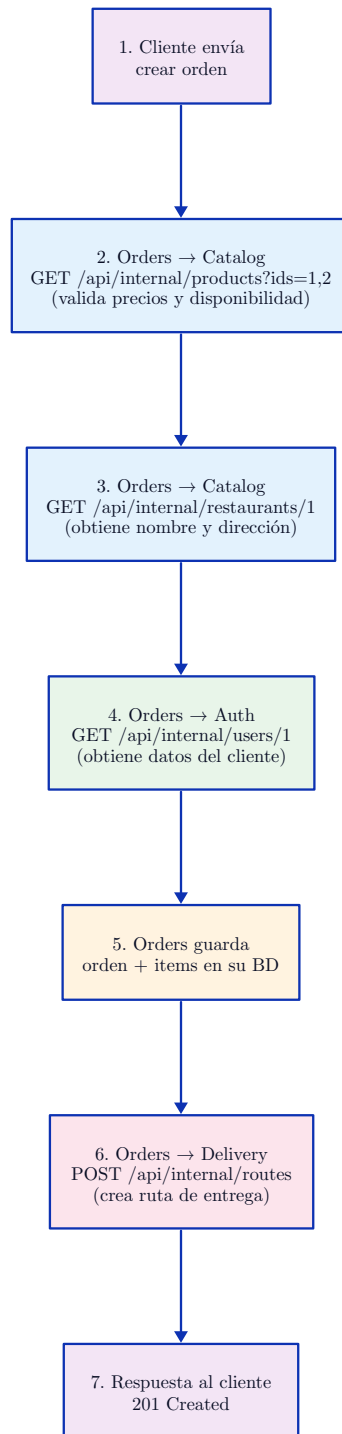
8.1 Interacciones por servicio con su BD



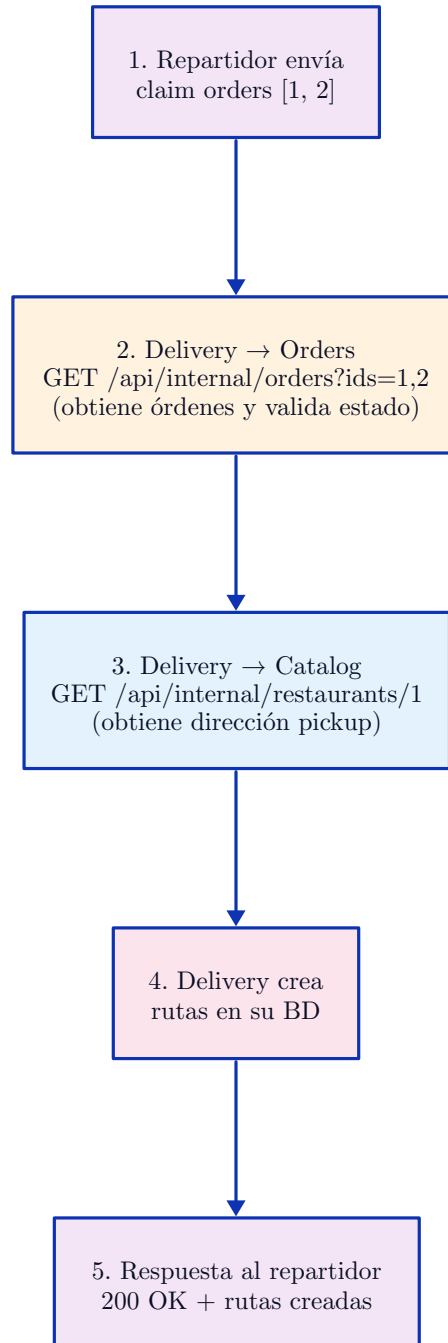
8.2 Llamadas HTTP entre servicios



8.3 Flujo completo: Crear una orden (secuencia)



8.4 Flujo completo: Repartidor reclama órdenes (secuencia)



Para evitar descoordinaciones entre quien expone y quien consume, estos son los **contratos exactos** de cada endpoint interno. Los nombres de campos, tipos y nullabilidad son obligatorios.

8.1 Auth Service (Nicolás expone → Felipe y Sebastián consumen)

GET `\api/internal/users/{userId}`

```
// Response 200
{
  "id": 1,
  "name": "Cliente",          // string | null
  "lastName": "Demo",        // string | null
  "email": "cliente@demo.cl", // string, nunca null
  "phone": "+5691111111"      // string | null
}

// Response 404
{
  "status": 404,
  "error": "NOT_FOUND",
  "message": "User not found with id: 99"
}
```

GET `\api/internal/users/{userId}/roles`

```
// Response 200 (siempre array, vacío si no tiene roles)
\[
  { "id": 1, "name": "Cliente" },
  { "id": 2, "name": "Restaurante" }
\]
```

8.2 Catalog Service (Javier expone → Felipe y Sebastián consumen)

GET `\api/internal/products?ids=1,2,3`

```
// Response 200 (siempre array, vacío si ningún ID existe)
\[
  {
    "id": 1,
    "restaurantId": 1,      // long, nunca null
    "name": "Burger doble", // string, nunca null
    "description": "Doble carne...", // string | null
    "price": 8990.00,        // decimal, nunca null, >= 0
    "image": "assets/img/...", // string | null
    "isAvailable": true      // boolean, nunca null
  }
\]
```

GET `\api/internal/restaurants/{restaurantId}`

```
// Response 200
{
  "id": 1,
```

```

    \"userId\": 2,                \\\\ long, nunca null
    \"name\": \"Urban Burger Demo\", \\\\ string, nunca null
    \"address\": \"Av. Providencia 1200\" \\\\ string | null
  }

// Response 404
{
  \"status\": 404,
  \"error\": \"NOT_FOUND\",
  \"message\": \"Restaurant not found with id: 99\"
}

```

GET `/api/internal/restaurants?userId={userId}`

Mismo formato de respuesta que `GET /api/internal/restaurants/{id}`. Retorna 404 si el usuario no tiene restaurante.

8.3 Orders Service (Felipe expone → Sebastián consume)

GET `/api/internal/orders?ids=1,2,3`

```

\\\\ Response 200 (siempre array)
\\[
  {
    \"id\": 1,
    \"clientId\": 1,                \\\\ long, nunca null
    \"restaurantId\": 1,           \\\\ long, nunca null
    \"deliveryId\": 1,             \\\\ long | null
    \"status\": \"Preparando\",     \\\\ string, nunca null
    \"address\": \"Av. Providencia 1200, Santiago\" \\\\ string, nunca null
  }
\\]

```

8.4 Delivery Service (Sebastián expone → Felipe consume)

GET `/api/internal/delivery-persons?userId={userId}`

```

\\\\ Response 200
{
  \"id\": 1,
  \"userId\": 3,                  \\\\ long, nunca null
  \"vehicle\": \"Moto\"           \\\\ string | null
}

// Response 404
{
  \"status\": 404,
  \"error\": \"NOT_FOUND\",
  \"message\": \"Delivery person not found for userId: 99\"
}

```

POST `/api/internal/routes`

```

\\ Request body
{
  \"orderId\": 1,                \\ long, requerido
  \"pickupAddress\": \"Av. Providencia 1000\", \\ string, requerido
  \"deliveryAddress\": \"Santa Isabel 060\",  \\ string, requerido
  \"distanceKm\": 2.4,           \\ decimal, requerido
  \"estimatedMinutes\": 18,      \\ int, requerido
  \"status\": \"Pendiente\"      \\ string, requerido
}

// Response 201
{
  \"id\": 1,
  \"orderId\": 1,
  \"pickupAddress\": \"Av. Providencia 1000\",
  \"deliveryAddress\": \"Santa Isabel 060\",
  \"distanceKm\": 2.4,
  \"estimatedMinutes\": 18,
  \"status\": \"Pendiente\"
}

// Response 409 (ruta ya existe para esa orden)
{
  \"status\": 409,
  \"error\": \"CONFLICT\",
  \"message\": \"Route already exists for orderId: 1\"
}

```

PATCH `/api/internal/routes/{orderId}/status`

```

\\ Request body
{
  \"status\": \"En camino\"  \\ string, requerido
}

// Response 200
{
  \"id\": 1,
  \"orderId\": 1,
  \"status\": \"En camino\"
}

```

Todos los servicios deben devolver errores con esta misma estructura. Esto permite que los HTTP clients de los consumidores tengan un solo `ErrorResponse` DTO compartido.

```

{
  \"status\": 404,                \\ int – HTTP status code
  \"error\": \"NOT_FOUND\",      \\ string – código de error
  constante
  \"message\": \"Resource not found with id: 99\"  \\ string – mensaje legible para

```

```
debug
}
```

Códigos de error estándar

HTTP Status	Campo <code>error</code>	Cuándo usarlo
400	<code>BAD_REQUEST</code>	Parámetros inválidos, body malformado
401	<code>UNAUTHORIZED</code>	Token faltante o inválido
403	<code>FORBIDDEN</code>	Token válido pero sin permisos
404	<code>NOT_FOUND</code>	Recurso no existe
409	<code>CONFLICT</code>	Recurso ya existe (ej: ruta duplicada)
422	<code>VALIDATION_ERROR</code>	Regla de negocio violada
500	<code>INTERNAL_ERROR</code>	Error inesperado del servidor
503	<code>SERVICE_UNAVAILABLE</code>	Servicio dependiente caído o timeout

Implementación recomendada

Cada servicio debe tener un `GlobalExceptionHandler` (ya lo tienen la mayoría) que mapee excepciones de dominio a este formato:

```
// DTO compartido – cada servicio crea su propia copia
public record ErrorResponse(int status, String error, String message) {}

// En el GlobalExceptionHandler
@OverrideHandler(ResourceNotFoundException.class)
public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException ex) {
    return ResponseEntity.status(404)
        .body(new ErrorResponse(404, "NOT\_FOUND", ex.getMessage()));
}
```

⚠ ATENCIÓN: Si el formato de error NO es consistente entre servicios, los HTTP clients van a necesitar parseo personalizado para cada servicio. Eso es exactamente lo que queremos evitar.

Cada servicio necesita saber la URL base de los servicios que consume. Estos son los nombres **exactos** de las variables que deben usar todos:

Variables estándar por servicio

```
\# \---- Base de datos propia \----
SUPABASE\_URL=https://xxxxx.supabase.co
SUPABASE\_SERVICE\_ROLE\_KEY=eyJhbG...

\# \---- URLs de otros servicios (solo las que necesita) \----
AUTH\_SERVICE\_URL=http://localhost:8081
CATALOG\_SERVICE\_URL=http://localhost:8082
ORDERS\_SERVICE\_URL=http://localhost:8083
DELIVERY\_SERVICE\_URL=http://localhost:8084

\# \---- Seguridad inter-servicio \----
INTERNAL\_API\_KEY=una-clave-compartida-entre-los-4-servicios
```

Qué variables necesita cada servicio

Variable	Nicolás (Auth)	Felipe (Orders)	Javier (Catalog)	Sebastián (Delivery)
SUPABASE_URL (propia)	✓	✓	✓	✓
SUPABASE_SERVICE_ROLE_KEY	✓	✓	✓	✓
AUTH_SERVICE_URL	—	✓	—	—
CATALOG_SERVICE_URL	—	✓	—	✓
ORDERS_SERVICE_URL	—	—	—	✓
DELIVERY_SERVICE_URL	—	✓	—	—
INTERNAL_API_KEY	✓	✓	✓	✓

Puertos locales asignados

Servicio	Puerto	Responsable
Auth	8081	Nicolás
Catalog	8082	Javier
Orders	8083	Felipe
Delivery	8084	Sebastián

Uso en `application.yml`

```
\# Ejemplo para Orders Service (Felipe)
services:
  auth:
    base-url: ${AUTH\_SERVICE\_URL:http://localhost:8081}
```



```

catalog:
  base-url: \${CATALOG\_SERVICE\_URL:http://localhost:8082}
delivery:
  base-url: \${DELIVERY\_SERVICE\_URL:http://localhost:8084}
internal-api-key: \${INTERNAL\_API\_KEY:dev-key}

```

Protección de endpoints internos

Todos los endpoints bajo `\api/internal/\*\*` deben validar el header `X-Internal-API-Key`:

```

// Filtro simple – cada servicio lo incluye
@Component
public class InternalApiKeyFilter extends OncePerRequestFilter {

    @Value("\${services.internal-api-key}")
    private String expectedKey;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain chain) throws ServletException,
    IOException {
        if (request.getRequestURI().startsWith("/api/internal/")) {
            String key = request.getHeader("X-Internal-API-Key");
            if (!expectedKey.equals(key)) {
                response.setStatus(403);
                response.getWriter().write("{\"status\":403,\"error\":\"\\\"FORBIDDEN\\\"\",\\\"message\\\":\\\"Invalid internal API key\\\"}");
                return;
            }
        }
        chain.doFilter(request, response);
    }
}

```

Cada servicio debe incluir los siguientes tests **antes de considerar su migración como terminada**. Se organizan en 3 niveles: unitarios (lógica pura), de integración (adaptadores HTTP) y de contrato (endpoints internos).

12.1 Nicolás — Auth Service

Tests unitarios (dominio y use cases):

- `RegisterUserTest` — Registrar usuario con email válido crea user + login + asigna rol
- `RegisterUserTest` — Email duplicado lanza `EmailAlreadyRegisteredException`
- `RegisterUserTest` — Password débil lanza `WeakPasswordException`
- `AuthenticateUserTest` — Credenciales válidas retornan JWT + refresh token
- `AuthenticateUserTest` — Credenciales inválidas lanzan `InvalidCredentialsException`
- `RefreshTokenTest` — Token de refresco válido genera nuevo JWT
- `RefreshTokenTest` — Token expirado o inválido lanza `InvalidTokenException`

Tests de integración (endpoints internos):

- [] GET `\api/internal/users/{id}` — Usuario existente retorna 200 con datos (name, lastName, email, phone)
- [] GET `\api/internal/users/{id}` — Usuario inexistente retorna 404
- [] GET `\api/internal/users/{id}/roles` — Retorna lista de roles asignados al usuario
- [] GET `\api/internal/users/{id}/roles` — Usuario sin roles retorna lista vacía

Test de contrato:

- [] El JSON de respuesta de `\api/internal/users/{id}` contiene exactamente los campos que Orders y Delivery esperan consumir: `id`, `name`, `lastName`, `email`, `phone`
-

12.2 Felipe — Orders Service

Tests unitarios (dominio y use cases):

- [] `CreateOrderTest` — Crear orden con productos válidos calcula subtotal, delivery_fee y total correctamente
- [] `CreateOrderTest` — Producto no disponible lanza excepción de dominio
- [] `CreateOrderTest` — Lista de productos vacía lanza excepción
- [] `UpdateOrderStatusTest` — Transición de estado válida (ej: "Nuevo pedido" → "Preparando") se aplica
- [] `UpdateOrderStatusTest` — Transición de estado inválida lanza excepción
- [] `ClaimOrdersTest` — Asignar repartidor a órdenes actualiza delivery_id y status

Tests de integración (HTTP clients hacia otros servicios):

- [] `CatalogHttpClientAdapter` — Llamada a Catalog con IDs válidos retorna lista de productos con precio
- [] `CatalogHttpClientAdapter` — Catalog caído o timeout lanza excepción manejable (no 500 genérico)
- [] `AuthHttpClientAdapter` — Llamada a Auth retorna datos del usuario
- [] `AuthHttpClientAdapter` — Usuario inexistente en Auth retorna `Optional.empty` (no excepción)
- [] `DeliveryHttpClientAdapter` — Llamada a Delivery retorna info del repartidor
- [] `DeliveryHttpClientAdapter` — Creación de ruta vía Delivery API retorna 201

 **TIP:** Usar `MockRestServiceServer` o `WireMock` para simular las respuestas de Auth, Catalog y Delivery sin necesidad de levantarlos.

Tests de integración (endpoint interno):

- [] GET `\api/internal/orders?ids\=1,2,3` — Retorna las órdenes existentes con sus campos
 - [] GET `\api/internal/orders?ids\=999` — IDs inexistentes retornan lista vacía
-

12.3 Javier — Catalog Service

Tests unitarios (dominio y use cases):

- [] `ListProductsTest` — Listar productos por categoría retorna solo productos de esa categoría
- [] `ListProductsTest` — Listar productos por restaurante retorna solo los del restaurante

- [] `CreateProductTest` — Crear producto con precio negativo lanza excepción de dominio
- [] `CreateProductTest` — Crear producto con categoría inexistente lanza excepción
- [] `GetProductsByIdsTest` — IDs válidos retornan productos; IDs inexistentes se ignoran
- [] `ListCategoriesTest` — Retorna todas las categorías ordenadas
- [] `ListRestaurantsTest` — Retorna todos los restaurantes

Tests de integración (endpoints internos):

- [] `GET \api/internal/products?ids\=1,2,3` — Retorna productos con campos: `id`, `restaurantId`, `name`, `description`, `price`, `image`, `isAvailable`
- [] `GET \api/internal/products?ids\=999` — IDs inexistentes retornan lista vacía
- [] `GET \api/internal/restaurants/{id}` — Restaurante existente retorna 200 con `id`, `userId`, `name`, `address`
- [] `GET \api/internal/restaurants/{id}` — Restaurante inexistente retorna 404
- [] `GET \api/internal/restaurants?userId\={id}` — Retorna restaurante del usuario o 404

Test de contrato:

- [] El JSON de `\api/internal/products?ids\=...` contiene exactamente los campos que Orders espera en `ProductInfo`: `id`, `restaurantId`, `name`, `description`, `price`, `image`, `isAvailable`
 - [] El JSON de `\api/internal/restaurants/{id}` contiene exactamente los campos que Orders y Delivery esperan en `RestaurantInfo`: `id`, `userId`, `name`, `address`
-

12.4 Sebastián — Delivery Service

Tests unitarios (dominio y use cases):

- [] `ClaimDeliveryTest` — Repartidor válido reclama órdenes del mismo restaurante exitosamente
- [] `ClaimDeliveryTest` — Órdenes de distintos restaurantes lanzan excepción
- [] `ClaimDeliveryTest` — Repartidor inexistente lanza `DeliveryPersonNotFoundException`
- [] `UpdateRouteStatusTest` — Transición de estado válida se aplica
- [] `UpdateRouteStatusTest` — Ruta inexistente lanza `RouteNotFoundException`
- [] `ListRoutesTest` — Retorna rutas del repartidor

Tests de integración (HTTP clients hacia otros servicios):

- [] `OrdersHttpClientAdapter` — Llamada a Orders API retorna órdenes con dirección y `restaurant_id`
- [] `OrdersHttpClientAdapter` — Orders caído lanza excepción manejable
- [] `CatalogHttpClientAdapter` — Llamada a Catalog retorna nombre y dirección del restaurante
- [] `CatalogHttpClientAdapter` — Restaurante inexistente retorna fallback (no rompe el flujo)

Tests de integración (endpoints internos):

- [] `GET \api/internal/delivery-persons?userId\={id}` — Repartidor existente retorna 200 con `id`, `userId`, `vehicle`
- [] `GET \api/internal/delivery-persons?userId\={id}` — Usuario sin perfil de repartidor retorna 404
- [] `POST \api/internal/routes` — Body válido crea ruta y retorna 201
- [] `POST \api/internal/routes` — Orden que ya tiene ruta lanza `RouteAlreadyAssignedException`

- [] PATCH `\api/internal/routes/{orderId}/status` — Actualiza estado y retorna 200
 - [] PATCH `\api/internal/routes/{orderId}/status` — Ruta inexistente retorna 404
-

Resumen de tests mínimos por persona

Persona	Unitarios	HTTP Clients	Endpoints Internos	Contrato	Total mín.
Nicolás	7	—	4	1	12
Felipe	6	6	2	—	14
Javier	7	—	5	2	14
Sebastián	6	4	6	—	16

Antes de dar por terminada la migración:

- [] Ningún servicio tiene en su código referencias directas (vía SQL o REST Supabase) a tablas que no le pertenecen
- [] Cada `.env` apunta a una URL de Supabase distinta
- [] Los endpoints internos están protegidos con `X-Internal-API-Key` y no son accesibles desde el cliente mobile
- [] Los formatos de error de todos los servicios siguen el estándar de la sección 10
- [] El flujo completo de usuario funciona end-to-end con las 4 BDs separadas
- [] Cada servicio tiene su script de seed independiente
- [] Todos los tests mínimos de la sección 12 pasan en verde
- [] Las respuestas JSON de los endpoints internos coinciden con los contratos de la sección 9